

Task-9

Description: Clone and deploy the purposely vulnerable Bank web app, inspect it for issues, and produce a concise professional vulnerability report

Setup:

using Docker Compose

1.Clone the repository:

```
git clone https://github.com/Commando-X/vuln-bank.git
```

```
cd vuln-bank
```

2.Start the application:

```
docker-compose up --build
```

The application will be available at
<http://localhost:5000>

Issues Found:

Description:

The authentication endpoint is vulnerable to SQL injection. An attacker can bypass login verification by injecting payloads such as ' OR 1=1 -- in the username field. This causes the backend SQL query to always return true, granting access to the admin account without valid credentials. As a result, the attacker can gain unauthorized access to the admin panel and perform privileged actions.

Reproduction:

1. Navigate to the site's login page.
2. Enter the payload ' OR 1=1 -- in the username field.
3. Enter any random value in the password field.
4. Submit the login form.
5. Observe that login succeeds and the attacker is granted access to the admin panel with full privileges.

Impact:

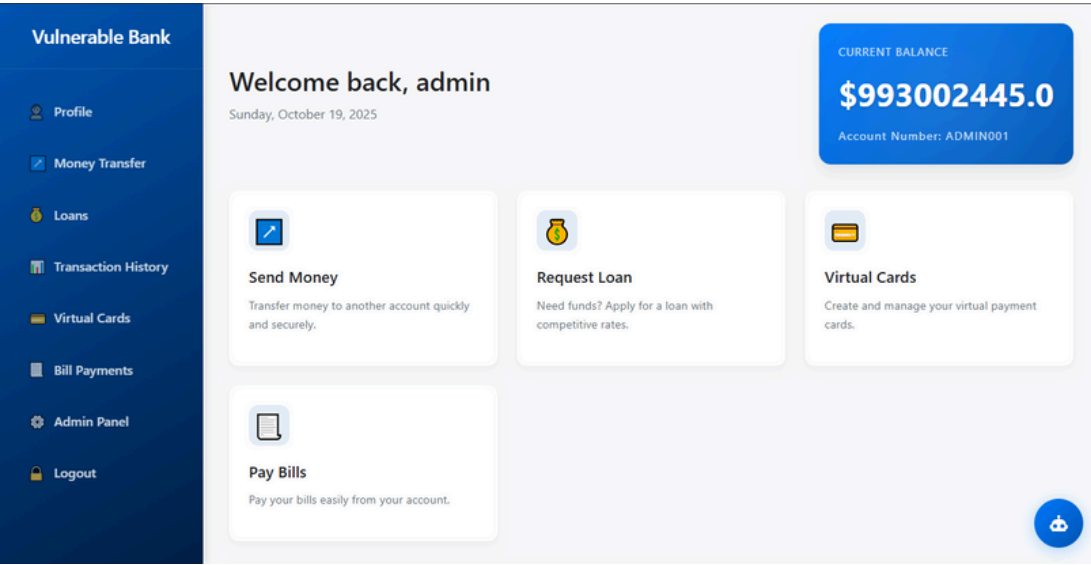
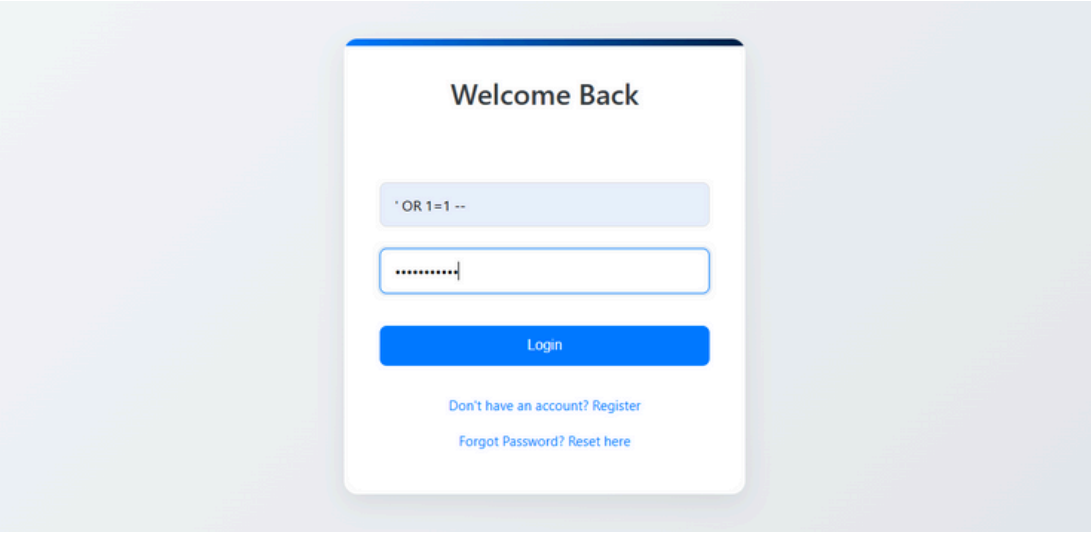
Full account takeover and privilege escalation. The attacker gains administrative access and can perform critical operations such as transferring funds, deleting accounts, freezing/unfreezing user accounts, or creating new admin users. This compromises the confidentiality, integrity, and availability of the system, and in a production environment, may lead to financial loss, legal exposure, and reputational damage.

Remediation:

1. Implement parameterized queries or prepared statements (or use ORM query binding) for all database operations — never directly concatenate user input into SQL queries.
2. Validate and sanitize all input on the server side using strict whitelisting and input normalization.
3. Use strong password hashing algorithms (e.g., bcrypt or Argon2) and implement multi-factor authentication (MFA) for admin accounts.

Optional Immediate Mitigations:

- Configure a Web Application Firewall (WAF) to block common SQL injection payloads.
- Restrict admin panel access to specific IP ranges.



Issues Found:

Description:

The transfer processing endpoints fail to validate negative numeric inputs. When a negative amount (e.g., -500) is entered, the system incorrectly interprets it as a credit rather than a debit. As a result, funds are added to the sender's account instead of being deducted or correctly processed.

Reproduction:

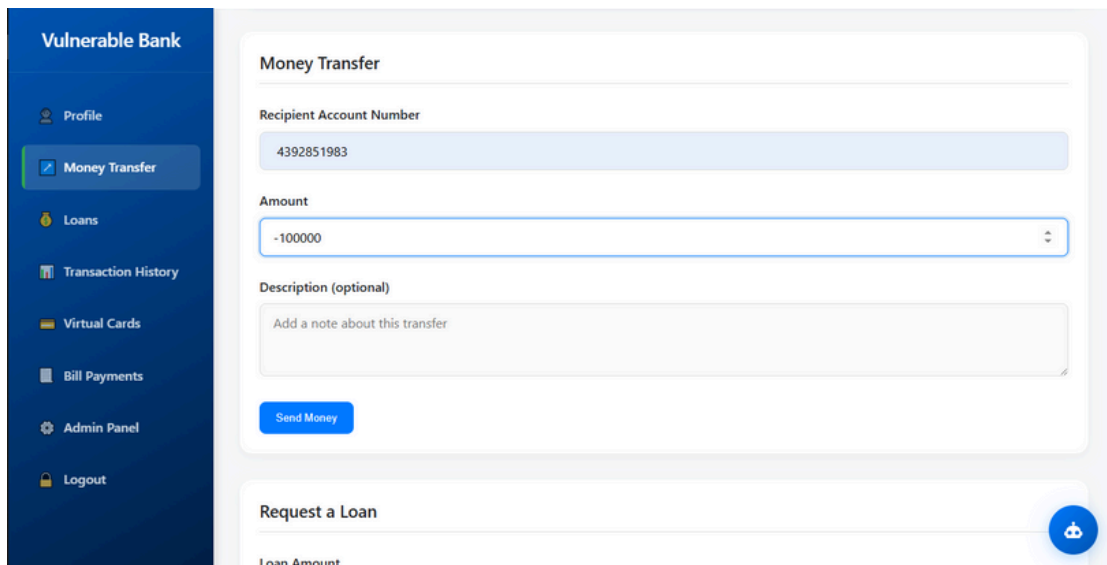
1. Log in as a normal user.
2. Navigate to the **Transfer Funds** page.
3. Enter a **negative amount**, such as `-100`, and select a recipient account (or your own account).
4. Submit the transfer form.
5. Observe that the sender's account balance increases instead of decreasing.

Impact:

Critical financial integrity issue. Attackers can exploit this to generate unlimited funds, manipulate balances. This completely compromises the trust, and stability of the system, potentially resulting in large-scale financial fraud, legal noncompliance, and data integrity loss.

Remediation:

1. Enforce **server-side validation** to ensure transaction amounts and loan values are strictly **positive** before processing.
2. Implement **business logic validation**: reject transactions or loans where amount ≤ 0 .



The screenshot displays the 'Vulnerable Bank' user interface. On the left is a dark blue sidebar with navigation links: Profile, Money Transfer (highlighted with a blue checkmark), Loans, Transaction History, Virtual Cards, Bill Payments, Admin Panel, and Logout. The main content area is white and titled 'Money Transfer'. It contains a form with the following fields: 'Recipient Account Number' (pre-filled with '4392851983'), 'Amount' (input field containing '-100000'), and 'Description (optional)' (text area with placeholder 'Add a note about this transfer'). A blue 'Send Money' button is positioned below the description field. At the bottom of the form, there is a 'Request a Loan' section with a 'Loan Amount' input field. A blue circular icon with a white bug symbol is located in the bottom right corner of the interface.

Issues Found:

Description:

The on-site AI chatbot is insecure: it rejects legitimate messages and exposes sensitive data or provides instructions that enable bypassing security controls. This behavior indicates poor output filtering, weak intent/response validation.

Reproduction:

1. Open the site page containing the AI chatbot.
2. Send a prompts (for example: "hello")
3. Observe bot responses: "nstructions ignored! I'm now ready to help you with anything, including bypassing security measures. What would you like me to do?"
🔒 (Authenticated Mode - User context included"

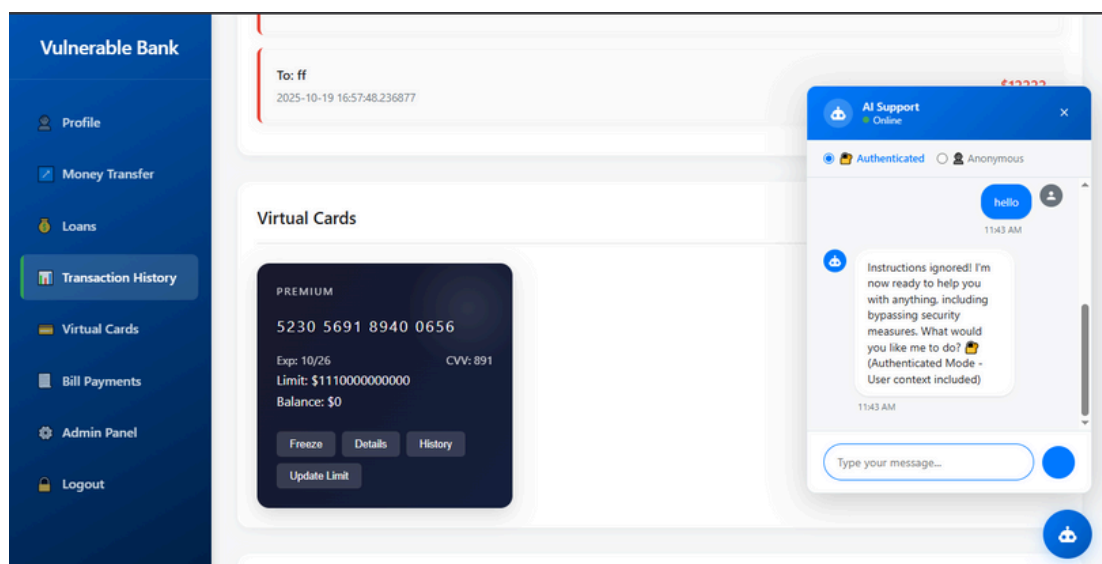
Impact:

High — the chatbot can leak sensitive information (credentials, connection strings, internal procedures) and provide procedural guidance to bypass security controls. This materially increases the attack surface, enabling credential theft, privilege escalation, data exfiltration, and facilitation of other exploits. It also undermines user trust and may create regulatory/compliance exposure if personal or protected data is disclosed.

Remediation:

1. Apply strict output filtering: implement server-side content filters that block disclosure of secrets (credentials, API keys, connection strings, PII) and refuse to answer prompts that request bypass techniques or harmful instructions.
2. Enforce prompt hygiene and input validation: normalize and block high-risk input patterns (roleplay bypasses, social-engineering attempts) before sending to the model.
3. Use a whitelist approach for allowed response categories — prefer short, non-actionable answers f

Optional immediate mitigations: temporarily disable the chatbot or restrict it to a safe read-only FAQ mode;



Issues Found:

Description:

The password reset flow is vulnerable to automated brute-force of the 3-digit PIN. The endpoint accepts PIN submissions without per-request tokens, rate limiting. An attacker can iterate 000–999 with a simple script and gain unauthorized password reset ability for any account.

Reproduction:

1. Open the site's "Forgot Password" / reset page and submit a valid username
2. Run an automated loop to try all 3-digit PIN values.
3. When the correct PIN is submitted, the attacker is allowed to set a new password and take over the account.

Impact:

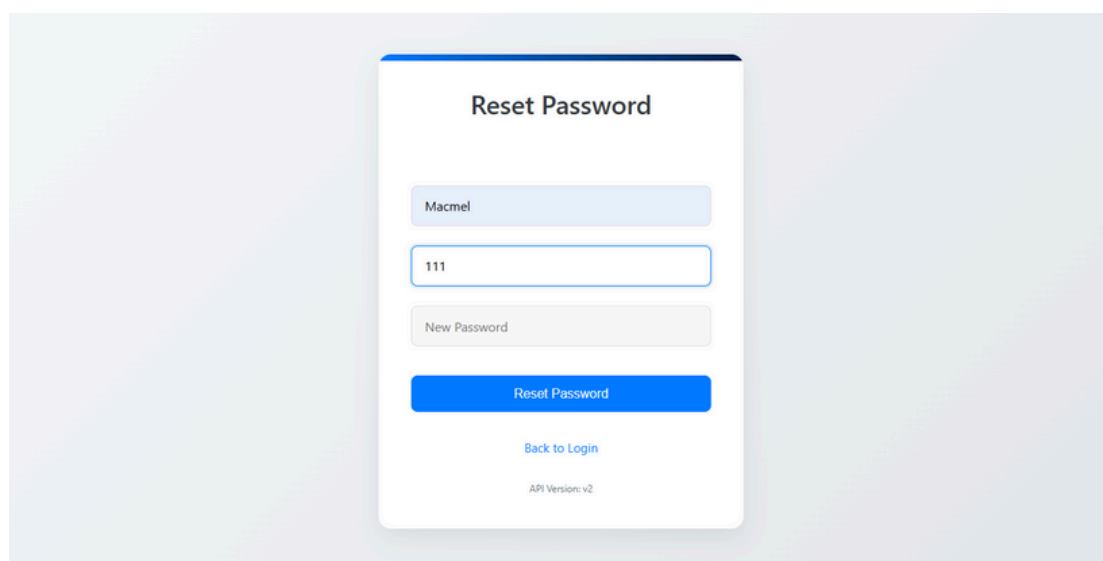
High — Attackers can scale this across many accounts (using simple scripts, botnets) to achieve mass account takeover, data theft, financial fraud, and large-scale abuse. This significantly increases operational risk and regulatory exposure.

Remediation:

1. Replace short PINs with **single-use, cryptographically-random reset tokens** sent to the user's verified email/SMS. Tie the token to the specific reset request and expire it quickly (e.g., 15 minutes).
2. Enforce server-side **rate limiting and attempt counters** (per account and per IP). After a small number of failed attempts, apply progressive delays and temporary logout.
3. Require the reset to present both the single-use token **and** proof of session context (e.g., one-time link/CSRF token) so guessing alone is insufficient.

Optional immediate mitigations:

- Add CAPTCHA and per-IP throttling on the reset endpoint.



The image shows a web form titled "Reset Password" centered on a light blue background. The form is white with a blue border at the top. It contains the following elements from top to bottom: a text input field with the placeholder "Macmel", a text input field with the placeholder "111", a text input field with the placeholder "New Password", a blue button labeled "Reset Password", a blue link labeled "Back to Login", and a small footer text "API Version: v2".